In [1]:
```
# Import All Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
# Load Data
fundamentals = pd.read_csv(r"C:\Users\Divya Sharma\OneDrive\Desktop\Capstone_NY\
Price_split_adj = pd.read_csv(r"C:\Users\Divya Sharma\OneDrive\Desktop\Capstone_
securies = pd.read_csv(r"C:\Users\Divya Sharma\OneDrive\Desktop\Capstone_NY\secu
```

In [3]:
```
# Columns
print(fundamentals.columns)
print(Price_split_adj.columns)
print(securies.columns)
```

```
Index(['Unnamed: 0', 'Ticker Symbol', 'Period Ending', 'Accounts Payable',
       'Accounts Receivable', 'Add'l income/expense items', 'After Tax ROE',
       'Capital Expenditures', 'Capital Surplus', 'Cash Ratio',
       'Cash and Cash Equivalents', 'Changes in Inventories', 'Common Stocks',
       'Cost of Revenue', 'Current Ratio', 'Deferred Asset Charges',
       'Deferred Liability Charges', 'Depreciation',
       'Earnings Before Interest and Tax', 'Earnings Before Tax',
       'Effect of Exchange Rate',
       'Equity Earnings/Loss Unconsolidated Subsidiary', 'Fixed Assets',
       'Goodwill', 'Gross Margin', 'Gross Profit', 'Income Tax',
       'Intangible Assets', 'Interest Expense', 'Inventory', 'Investments',
       'Liabilities', 'Long-Term Debt', 'Long-Term Investments',
       'Minority Interest', 'Misc. Stocks', 'Net Borrowings', 'Net Cash Flow',
       'Net Cash Flow-Operating', 'Net Cash Flows-Financing',
       'Net Cash Flows-Investing', 'Net Income', 'Net Income Adjustments',
       'Net Income Applicable to Common Shareholders',
       'Net Income-Cont. Operations', 'Net Receivables', 'Non-Recurring Items',
       'Operating Income', 'Operating Margin', 'Other Assets',
       'Other Current Assets', 'Other Current Liabilities', 'Other Equity',
       'Other Financing Activities', 'Other Investing Activities',
       'Other Liabilities', 'Other Operating Activities',
       'Other Operating Items', 'Pre-Tax Margin', 'Pre-Tax ROE',
       'Profit Margin', 'Quick Ratio', 'Research and Development',
       'Retained Earnings', 'Sale and Purchase of Stock',
       'Sales, General and Admin.',
       'Short-Term Debt / Current Portion of Long-Term Debt',
       'Short-Term Investments', 'Total Assets', 'Total Current Assets',
       'Total Current Liabilities', 'Total Equity', 'Total Liabilities',
       'Total Liabilities & Equity', 'Total Revenue', 'Treasury Stock',
       'For Year', 'Earnings Per Share', 'Estimated Shares Outstanding'],
      dtype='str')
Index(['date', 'symbol', 'open', 'close', 'low', 'high', 'volume'], dtype='str')
Index(['Ticker symbol', 'Security', 'SEC filings', 'GICS Sector',
       'GICS Sub Industry', 'Address of Headquarters', 'Date first added',
       'CIK'],
      dtype='str')
```

In [4]:
```
# Check data
Price_split_adj.head()
fundamentals.head()
securies.head()
```

Out[4]:

	Ticker symbol	Security	SEC filings	GICS Sector	GICS Sub Industry	Address of Headquarters	Date first added	
0	MMM	3M Company	reports	Industrials	Industrial Conglomerates	St. Paul, Minnesota	NaN	6
1	ABT	Abbott Laboratories	reports	Health Care	Health Care Equipment	North Chicago, Illinois	1964-03-31	
2	ABBV	AbbVie	reports	Health Care	Pharmaceuticals	North Chicago, Illinois	2012-12-31	155
3	ACN	Accenture plc	reports	Information Technology	IT Consulting & Other Services	Dublin, Ireland	2011-07-06	146
4	ATVI	Activision Blizzard	reports	Information Technology	Home Entertainment Software	Santa Monica, California	2015-08-31	71

In [5]:

```
Price_split_adj.info()

fundamentals.info()

securies.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 851264 entries, 0 to 851263
Data columns (total 7 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   date    851264 non-null  str
 1   symbol  851264 non-null  str
 2   open    851264 non-null  float64
 3   close   851264 non-null  float64
 4   low     851264 non-null  float64
 5   high    851264 non-null  float64
 6   volume  851264 non-null  float64
dtypes: float64(5), str(2)
memory usage: 45.5 MB
<class 'pandas.DataFrame'>
RangeIndex: 1781 entries, 0 to 1780
Data columns (total 79 columns):
 #   Column                                            Non-Null Count  Dtype
---  ------                                            --------------  -----
 0   Unnamed: 0                                        1781 non-null   int64
 1   Ticker Symbol                                     1781 non-null   str
 2   Period Ending                                     1781 non-null   str
 3   Accounts Payable                                  1781 non-null   float64
 4   Accounts Receivable                               1781 non-null   float64
 5   Add'l income/expense items                        1781 non-null   float64
 6   After Tax ROE                                     1781 non-null   float64
 7   Capital Expenditures                              1781 non-null   float64
 8   Capital Surplus                                   1781 non-null   float64
 9   Cash Ratio                                        1482 non-null   float64
 10  Cash and Cash Equivalents                         1781 non-null   float64
 11  Changes in Inventories                            1781 non-null   float64
 12  Common Stocks                                     1781 non-null   float64
 13  Cost of Revenue                                   1781 non-null   float64
 14  Current Ratio                                     1482 non-null   float64
 15  Deferred Asset Charges                            1781 non-null   float64
 16  Deferred Liability Charges                        1781 non-null   float64
 17  Depreciation                                      1781 non-null   float64
 18  Earnings Before Interest and Tax                  1781 non-null   float64
 19  Earnings Before Tax                               1781 non-null   float64
 20  Effect of Exchange Rate                           1781 non-null   float64
 21  Equity Earnings/Loss Unconsolidated Subsidiary    1781 non-null   float64
 22  Fixed Assets                                      1781 non-null   float64
 23  Goodwill                                          1781 non-null   float64
 24  Gross Margin                                      1781 non-null   float64
 25  Gross Profit                                      1781 non-null   float64
 26  Income Tax                                        1781 non-null   float64
 27  Intangible Assets                                 1781 non-null   float64
 28  Interest Expense                                  1781 non-null   float64
 29  Inventory                                         1781 non-null   float64
 30  Investments                                       1781 non-null   float64
 31  Liabilities                                       1781 non-null   float64
 32  Long-Term Debt                                    1781 non-null   float64
 33  Long-Term Investments                             1781 non-null   float64
 34  Minority Interest                                 1781 non-null   float64
 35  Misc. Stocks                                      1781 non-null   float64
 36  Net Borrowings                                    1781 non-null   float64
 37  Net Cash Flow                                     1781 non-null   float64
 38  Net Cash Flow-Operating                           1781 non-null   float64
 39  Net Cash Flows-Financing                          1781 non-null   float64
 40  Net Cash Flows-Investing                          1781 non-null   float64
```

```
 41  Net Income                                          1781 non-null   float64
 42  Net Income Adjustments                              1781 non-null   float64
 43  Net Income Applicable to Common Shareholders        1781 non-null   float64
 44  Net Income-Cont. Operations                         1781 non-null   float64
 45  Net Receivables                                     1781 non-null   float64
 46  Non-Recurring Items                                 1781 non-null   float64
 47  Operating Income                                    1781 non-null   float64
 48  Operating Margin                                    1781 non-null   float64
 49  Other Assets                                        1781 non-null   float64
 50  Other Current Assets                                1781 non-null   float64
 51  Other Current Liabilities                           1781 non-null   float64
 52  Other Equity                                        1781 non-null   float64
 53  Other Financing Activities                          1781 non-null   float64
 54  Other Investing Activities                          1781 non-null   float64
 55  Other Liabilities                                   1781 non-null   float64
 56  Other Operating Activities                          1781 non-null   float64
 57  Other Operating Items                               1781 non-null   float64
 58  Pre-Tax Margin                                      1781 non-null   float64
 59  Pre-Tax ROE                                         1781 non-null   float64
 60  Profit Margin                                       1781 non-null   float64
 61  Quick Ratio                                         1482 non-null   float64
 62  Research and Development                            1781 non-null   float64
 63  Retained Earnings                                   1781 non-null   float64
 64  Sale and Purchase of Stock                          1781 non-null   float64
 65  Sales, General and Admin.                           1781 non-null   float64
 66  Short-Term Debt / Current Portion of Long-Term Debt 1781 non-null   float64
 67  Short-Term Investments                              1781 non-null   float64
 68  Total Assets                                        1781 non-null   float64
 69  Total Current Assets                                1781 non-null   float64
 70  Total Current Liabilities                           1781 non-null   float64
 71  Total Equity                                        1781 non-null   float64
 72  Total Liabilities                                   1781 non-null   float64
 73  Total Liabilities & Equity                          1781 non-null   float64
 74  Total Revenue                                       1781 non-null   float64
 75  Treasury Stock                                      1781 non-null   float64
 76  For Year                                            1608 non-null   float64
 77  Earnings Per Share                                  1562 non-null   float64
 78  Estimated Shares Outstanding                        1562 non-null   float64
dtypes: float64(76), int64(1), str(2)
memory usage: 1.1 MB
<class 'pandas.DataFrame'>
RangeIndex: 505 entries, 0 to 504
Data columns (total 8 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Ticker symbol           505 non-null    str
 1   Security                505 non-null    str
 2   SEC filings             505 non-null    str
 3   GICS Sector             505 non-null    str
 4   GICS Sub Industry       505 non-null    str
 5   Address of Headquarters 505 non-null    str
 6   Date first added        307 non-null    str
 7   CIK                     505 non-null    int64
dtypes: int64(1), str(7)
memory usage: 31.7 KB
```

In [6]:
```
# Converting Str dates in Datetime
Price_split_adj['date'] = pd.to_datetime(Price_split_adj['date'])
fundamentals['Period Ending'] = pd.to_datetime(fundamentals['Period Ending'])
securies['Date first added'] = pd.to_datetime(securies['Date first added'])
```

In [7]:
```
# Year extract
fundamentals['Year'] = fundamentals['Period Ending'].dt.year
```

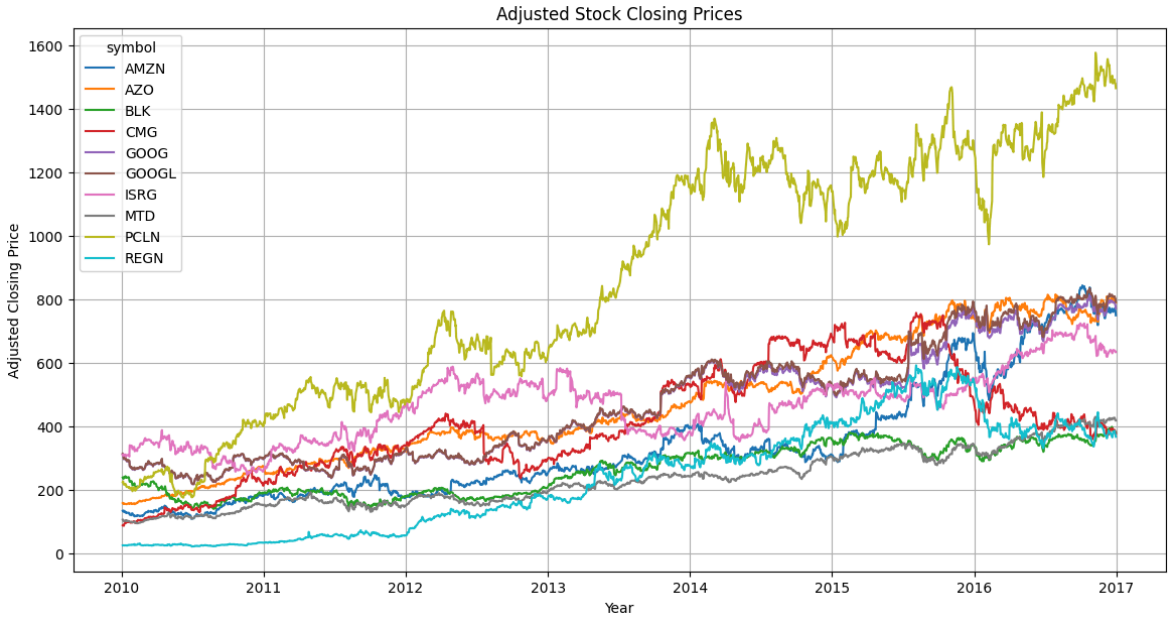
In [8]:
```
# Remove unnecessary column
fundamentals.drop('Unnamed: 0', axis=1, inplace=True)
```

PART 1 — EDA

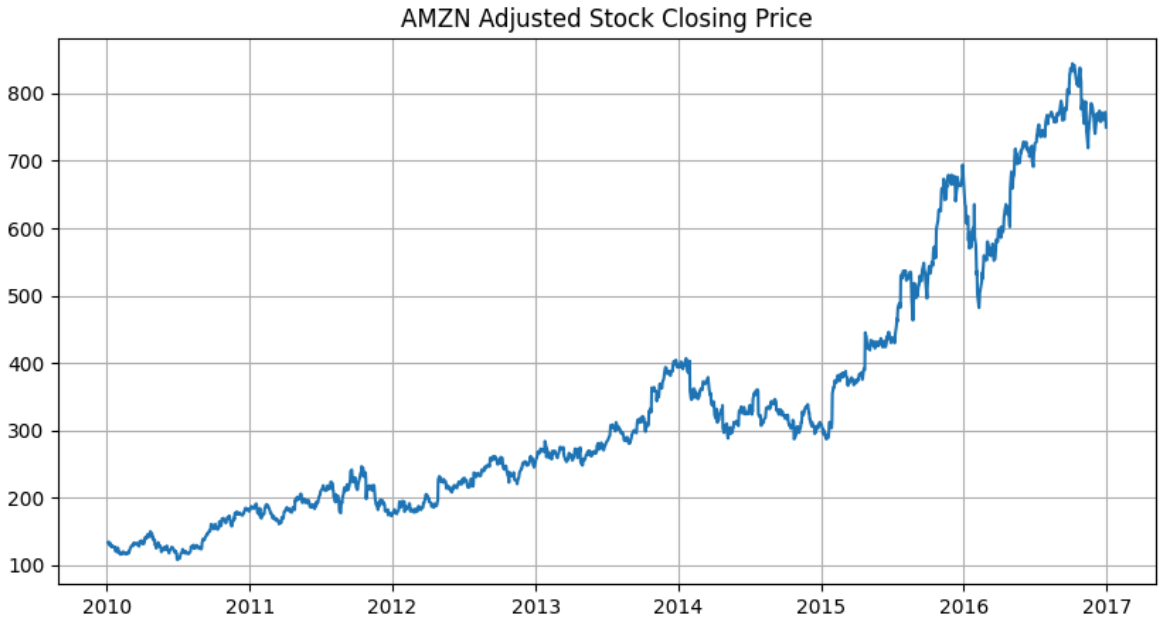
In [9]:
```
# 1 ADJUSTED STOCK PRICES
top10_symbols = Price_split_adj.groupby("symbol")["close"].mean().sort_values(as
```

In [10]:
```
top_10_Prices = Price_split_adj[Price_split_adj["symbol"].isin(top10_symbols)]
```

In [11]:
```
# Plot Adjusted Closing Prices (Top 10 Companies)
plt.figure(figsize=(14,7))

sns.lineplot(
    data = top_10_Prices,
    x='date',
    y='close',
    hue='symbol'
)


plt.title('Adjusted Stock Closing Prices')
plt.xlabel('Year')
plt.ylabel('Adjusted Closing Price')

plt.grid(True)

plt.show()
```

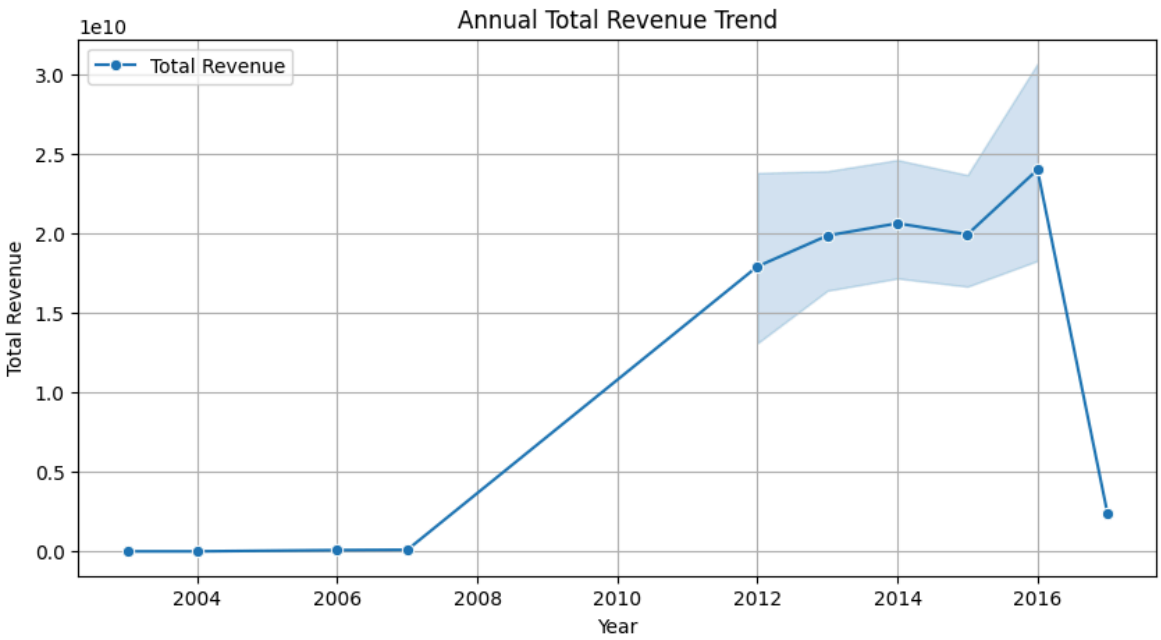


In [12]:
```
# ADJ CLOSING PRICE TREND OF 'AMZN' (One Company)

apple = Price_split_adj[Price_split_adj['symbol'] == 'AMZN']
plt.figure(figsize=(10,5))
plt.plot(apple['date'], apple['close'])
plt.title("AMZN Adjusted Stock Closing Price")

plt.grid(True)
plt.show()
```

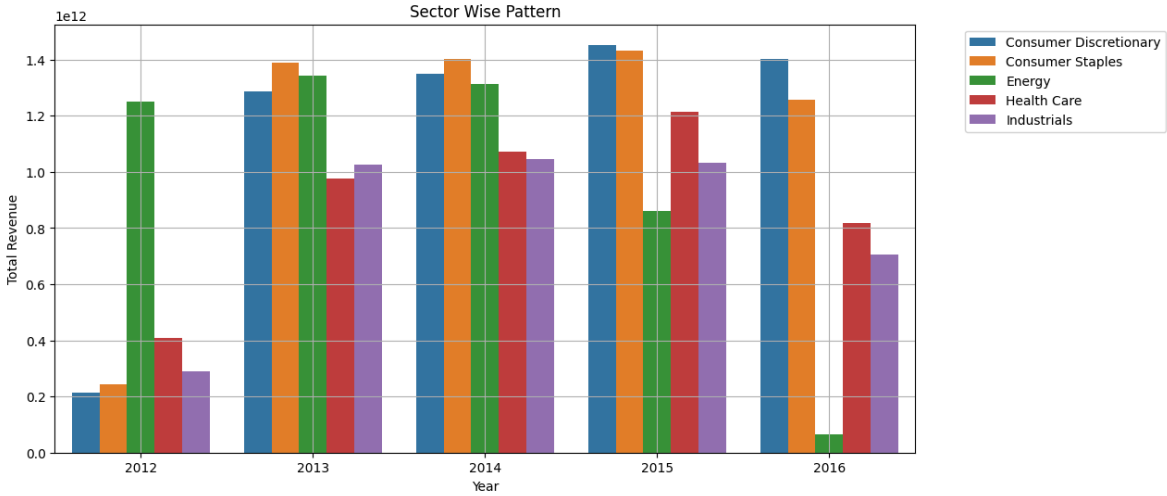


In [13]:
```
# 2 Financial Metrics over Years
```

In [14]:
```
plt.figure(figsize=(10,5))

sns.lineplot(
    data = fundamentals,
    x='Year',
    y='Total Revenue',
    marker="o",
    label="Total Revenue"
)

plt.title('Annual Total Revenue Trend')

plt.xlabel('Year')
plt.ylabel('Total Revenue')

plt.grid(True)

plt.show()
```

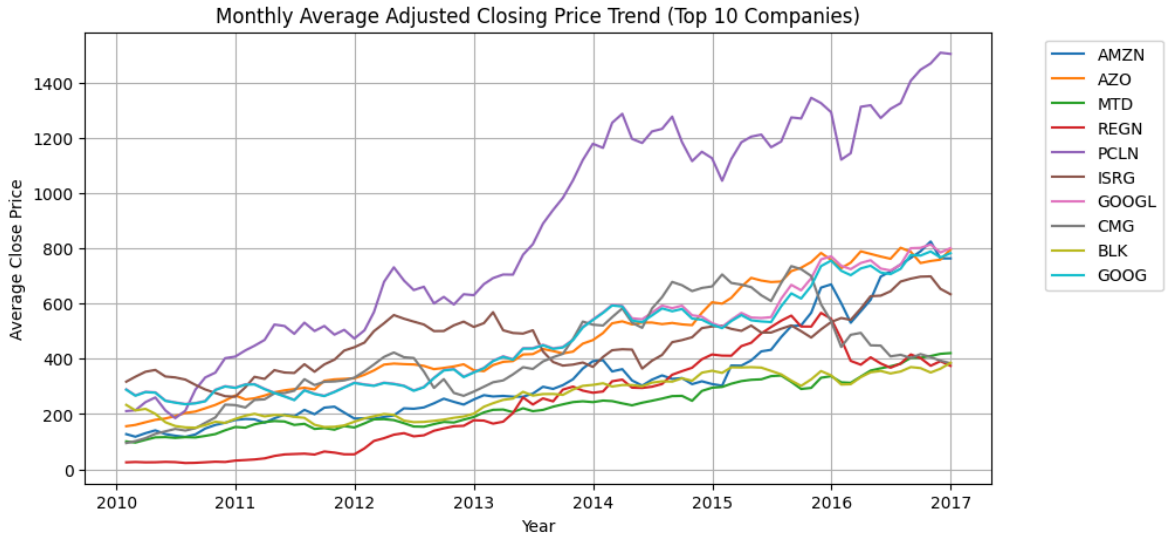


In [15]:
```
#3 Sector-wise Patterns

# First we merge sector
fund_sec = fundamentals.merge(securies, left_on="Ticker Symbol", right_on="Ticke

# Filter valid years
fund_sect = fund_sec[fund_sec["Year"].between(2012, 2016)]

# Sector revenue (Group by Sector and Year)
sector_rev = fund_sect.groupby(["Year", "GICS Sector"])["Total Revenue"].sum().r

# # Groupby & Top 5
top_sectors = (sector_rev.groupby("GICS Sector")["Total Revenue"].sum().nlargest

# Filter top sectors
sector_top = sector_rev[sector_rev["GICS Sector"].isin(top_sectors)]

# Convert year to string for clean axis
sector_rev["Year"] = sector_rev["Year"].astype(str)

# Plot - Total Revenue grouped by GICS Sector over years

plt.figure(figsize=(12,6))

sns.barplot(
    data=sector_top,
    x="Year",
    y="Total Revenue",
    hue="GICS Sector"
)

plt.title('Sector Wise Pattern')
plt.grid(True)

plt.legend(bbox_to_anchor=(1.05,1), loc="upper left")

plt.show()
```

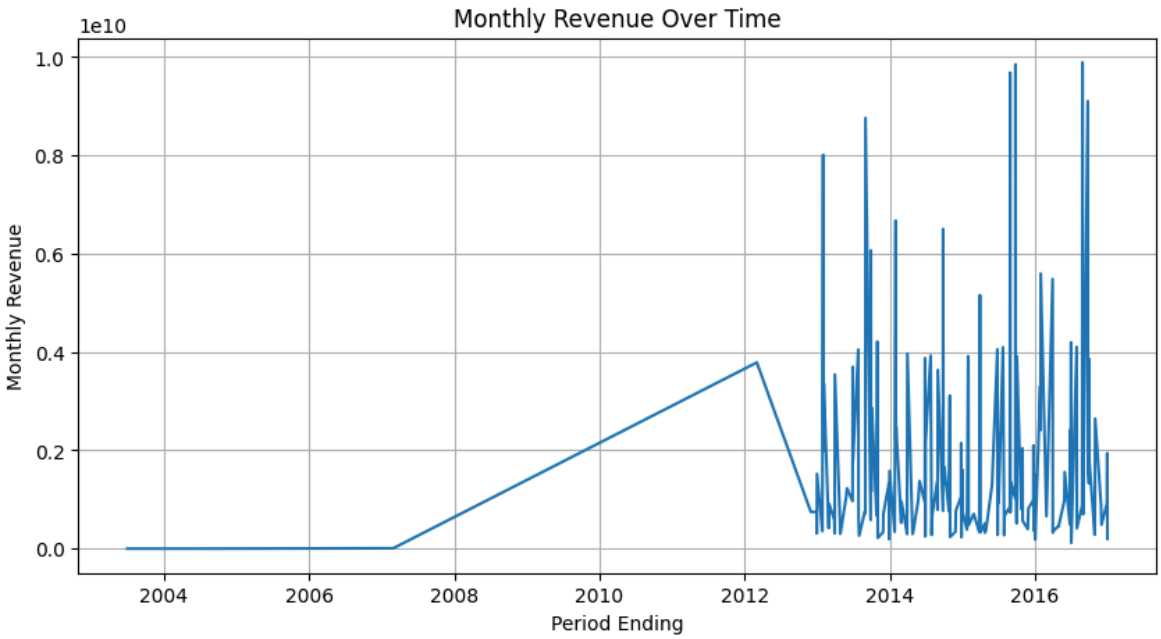


Part 2 Time series data creation

In [16]:
```
# 1 Monthly average adjusted closing prices

# Create Monthly Averages
monthly_price = Price_split_adj.groupby(['symbol', pd.Grouper(key='date', freq='

# Identify Top 10 Companies
top10 = Price_split_adj.groupby('symbol')['close'].mean().sort_values(ascending=

# Filter Monthly Data for Top 10
top10_prices = monthly_price[monthly_price['symbol'].isin(top10)]
top10_prices = top10_prices.sort_values('date')

plt.figure(figsize=(10,5))

sns.lineplot(
    data=top10_prices,
    x='date',
    y='close',
    hue='symbol'
)

plt.title('Monthly Average Adjusted Closing Price Trend (Top 10 Companies)')

plt.xlabel('Year')
plt.ylabel('Average Close Price')

plt.legend(bbox_to_anchor=(1.05,1), loc='upper left')

plt.grid(True)

plt.show()
```

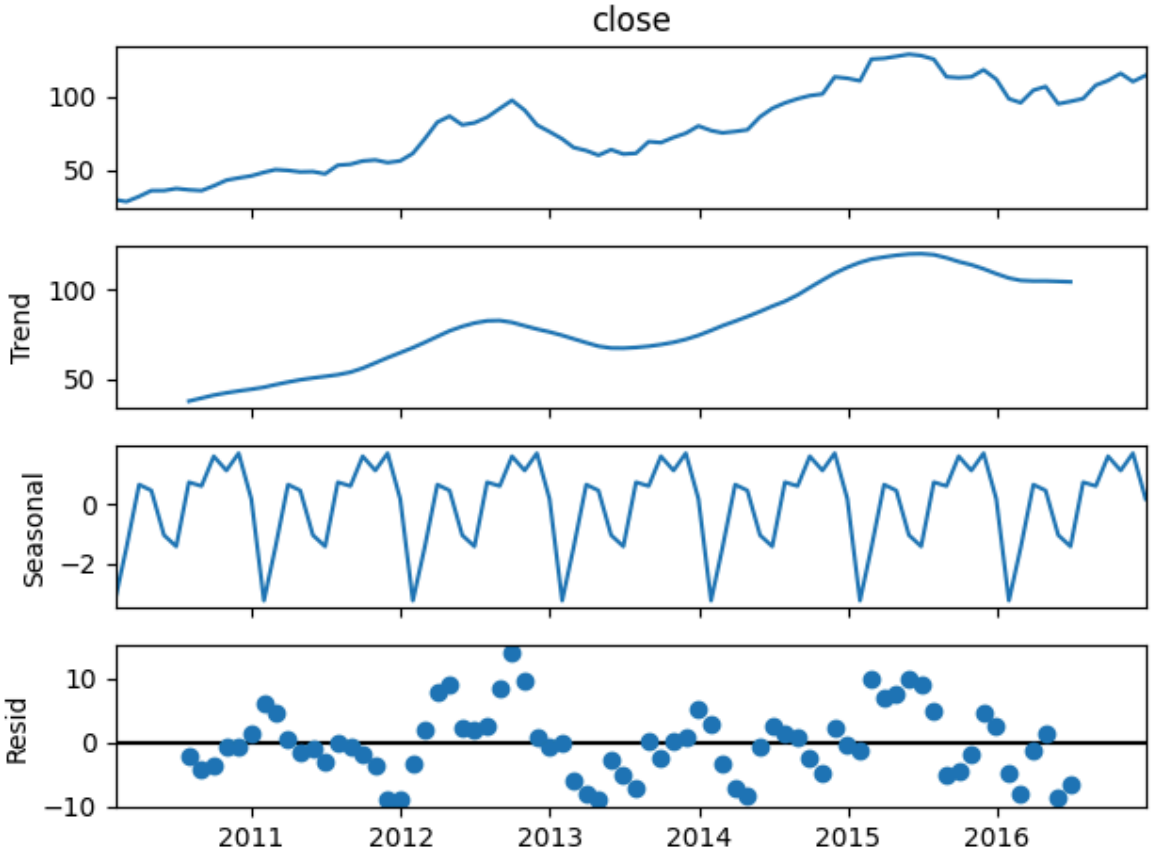


In [17]:
```
# 2.  Monthly Revenue (Using Fundamentals)

fundamentals["monthly_revenue"] = fundamentals["Total Revenue"] / 12

monthly_revenue = fundamentals.groupby( "Period Ending")["monthly_revenue"].mean

plt.figure(figsize=(10, 5))

sns.lineplot(data=monthly_revenue,
             x="Period Ending",
             y="monthly_revenue")

plt.title("Monthly Revenue Over Time")

plt.xlabel("Period Ending")

plt.ylabel("Monthly Revenue")

plt.grid(True)
plt.show()
```



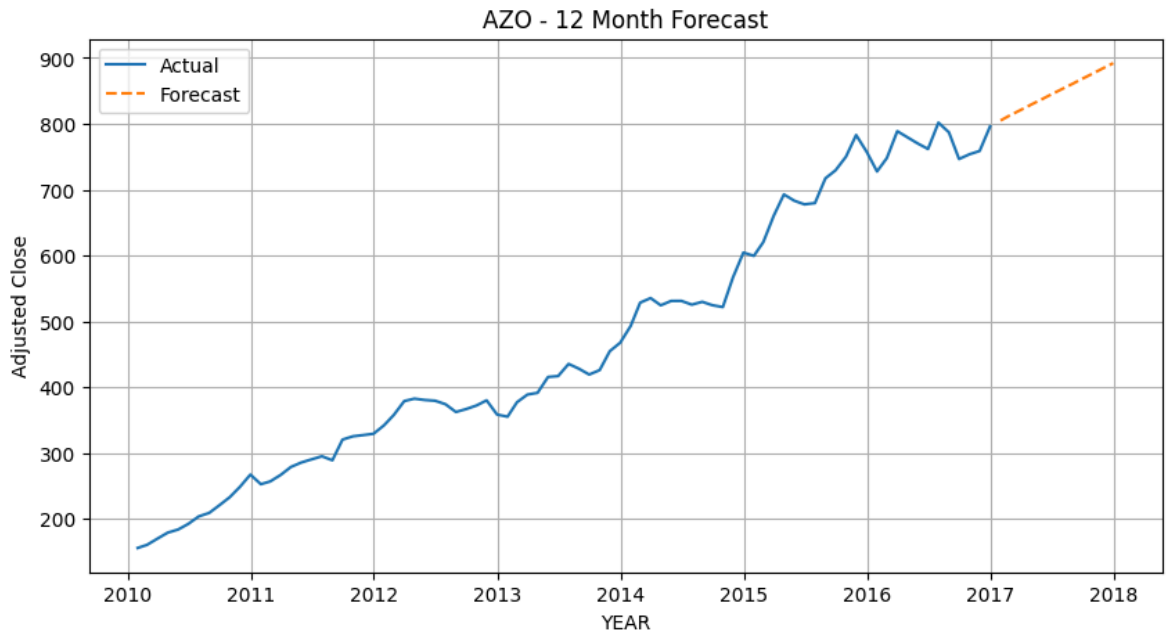
Part 3 Trend and Seasonality

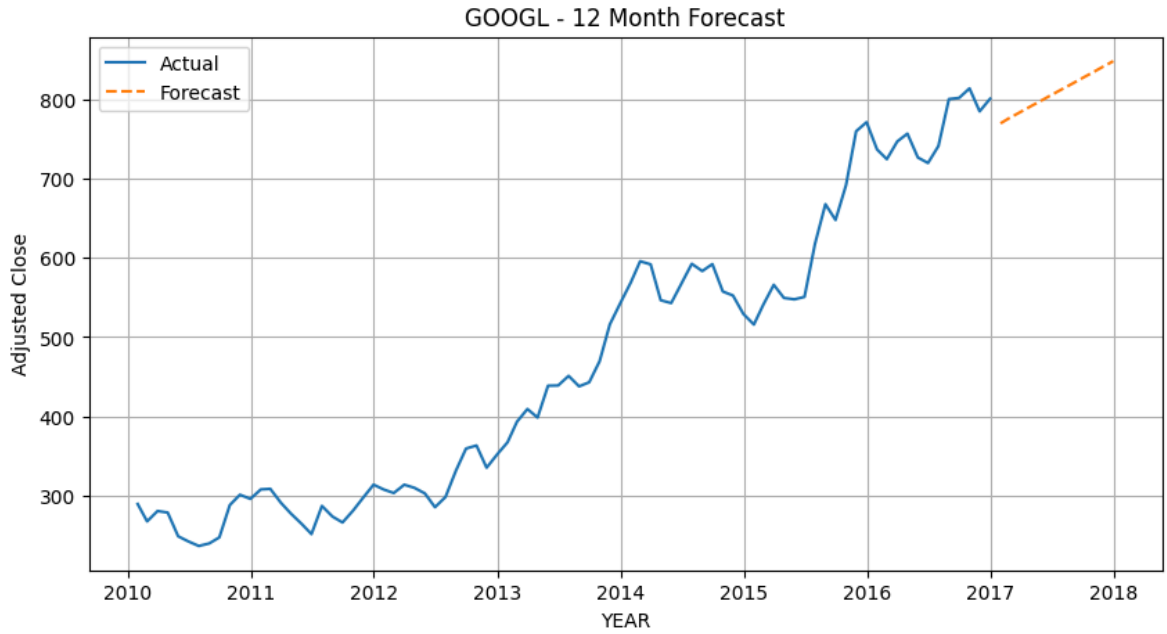
In [18]:
```
# 1. Use statsmodels or Prophet to detect seasonality in adjusted close prices -

# Select One Company (Original Price)
aapl = monthly_price[monthly_price['symbol']=='AAPL']

# set date as Index
aapl = aapl.set_index('date')

# Import Deccomposition
from statsmodels.tsa.seasonal import seasonal_decompose

# Apply Decomposition
result = seasonal_decompose(aapl['close'],model='additive',period=12)

result.plot()

plt.show()
```

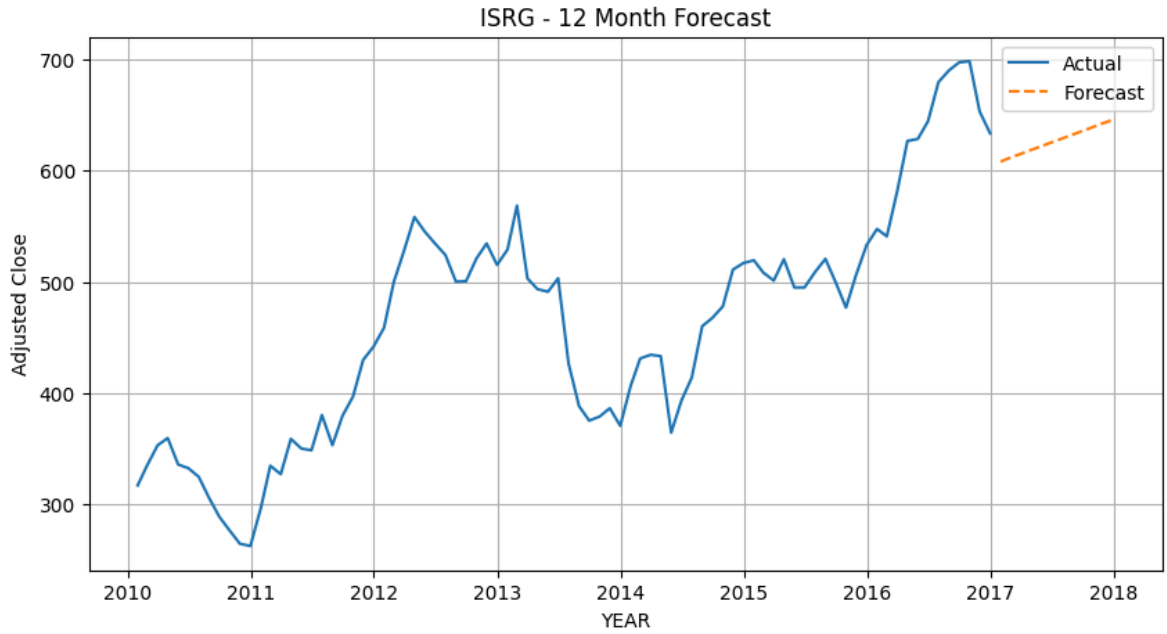


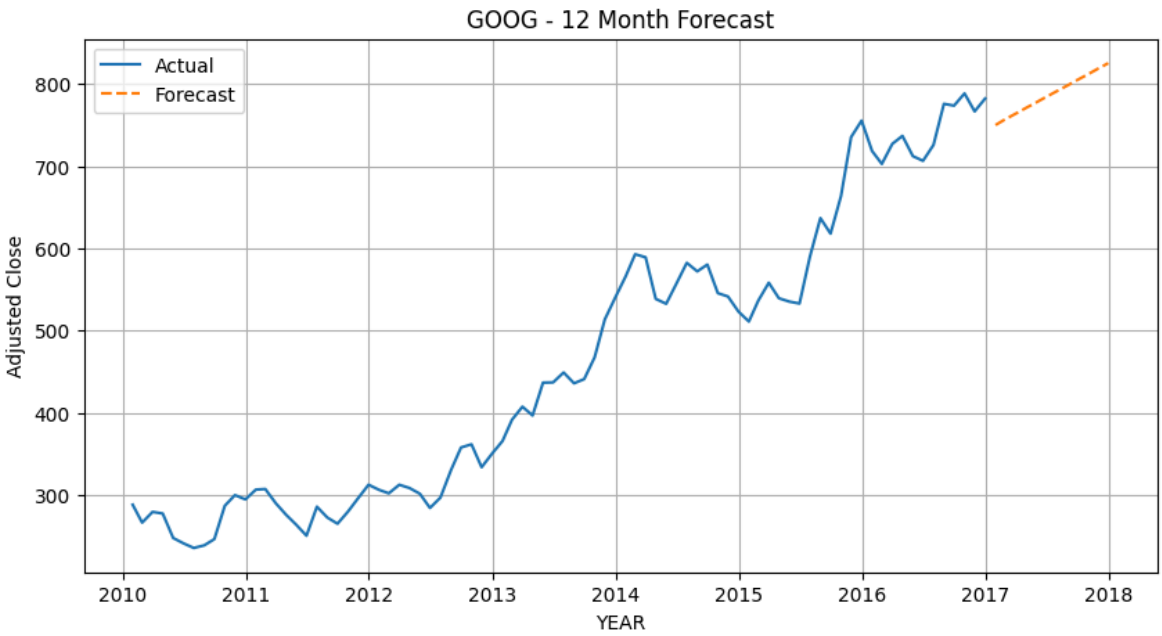
In [19]:
```
# 2. Forecast next 12 months of adjusted price for top 5 companies - Used Linear

from sklearn.linear_model import LinearRegression

top5 = (Price_split_adj.groupby('symbol')['close'].mean().sort_values(ascending=

for company in top5:

    company_data = monthly_price[monthly_price['symbol'] == company].copy()

    company_data = company_data.sort_values('date')

    company_data['Month_Num'] = range(len(company_data))
```

```
X = company_data[['Month_Num']]
y = company_data['close']

model = LinearRegression()
model.fit(X, y)

future_months = pd.DataFrame({
    'Month_Num': range(len(company_data), len(company_data)+12)
})

forecast = model.predict(future_months)

future_dates = pd.date_range(
    start=company_data['date'].iloc[-1] + pd.offsets.MonthEnd(1),
    periods=12,
    freq='ME'
)

plt.figure(figsize=(10,5))

plt.plot(company_data['date'], company_data['close'], label='Actual')
plt.plot(future_dates, forecast, label='Forecast', linestyle="--")

plt.title(f'{company} - 12 Month Forecast')
plt.xlabel('YEAR')
plt.ylabel('Adjusted Close')

plt.legend()
plt.grid(True)

plt.show()
```

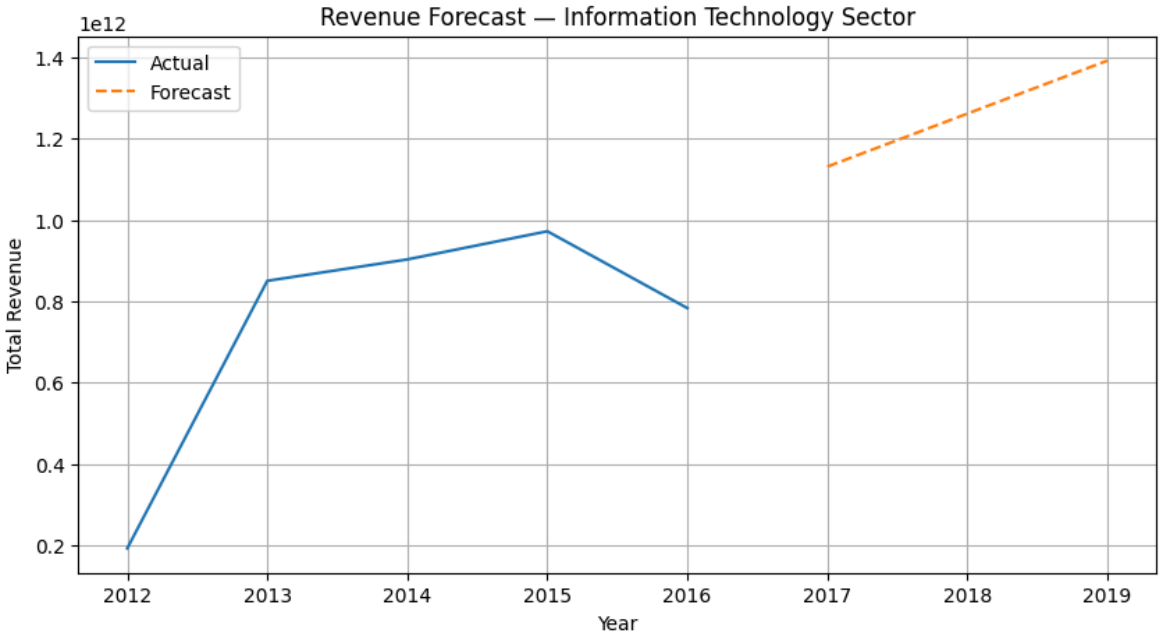

AZO - 12 Month Forecast
Actual
Forecast
Adjusted Close
900
800
700
600
500
400
300
200
2010
2011
2012
2013
2014
2015
2016
2017
2018
YEAR


GOOGL - 12 Month Forecast
Actual
Forecast
Adjusted Close
800
700
600
500
400
300
2010
2011
2012
2013
2014
2015
2016
2017
2018
YEAR


ISRG - 12 Month Forecast
Actual
Forecast
Adjusted Close
700
600
500
400
300
2010
2011
2012
2013
2014
2015
2016
2017
2018
YEAR



Part 4 - Category-Level Forecasting

In [20]:
```
# 1. Revenue forecast by GICS Sector

from sklearn.linear_model import LinearRegression

sector_data = fundamentals.merge(
    securies[['Ticker symbol', 'GICS Sector']],
    left_on='Ticker Symbol',
    right_on='Ticker symbol',
    how='left'
)

sector_data['Year'] = sector_data['Period Ending'].dt.year.astype(int)

sector_rev = (
    sector_data.groupby(['GICS Sector', 'Year'])['Total Revenue']
    .sum()
    .reset_index()
)

it_sector = sector_rev[
    sector_rev['GICS Sector'] == 'Information Technology'
].copy()

it_sector['Year_Num'] = range(len(it_sector))

X = it_sector[['Year_Num']]
y = it_sector['Total Revenue']

model = LinearRegression()
model.fit(X, y)

future_years = pd.DataFrame({
    'Year_Num': range(len(it_sector), len(it_sector)+3)
})

forecast = model.predict(future_years)
```

```
future_labels = range(
    it_sector['Year'].max()+1,
    it_sector['Year'].max()+4
)

plt.figure(figsize=(10,5))

plt.title('Revenue Forecast — Information Technology Sector')
plt.xlabel('Year')
plt.ylabel('Total Revenue')

plt.plot(it_sector['Year'], it_sector['Total Revenue'], label='Actual')
plt.plot(future_labels, forecast, label='Forecast', linestyle="--")

plt.legend()
plt.grid(True)

plt.show()
```